CACHING & REDIS

Backend Engineering from First Principles

Lecture 13 — Large-Font Study Notes

Caching fundamentals • CDN • DNS • CPU caches • Redis • Cache strategies • Eviction • Backend use cases

Detailed, interview-oriented notes based on the supplied lecture transcript.

Learning outcomes

- Explain why caching reduces latency, repeated computation and source-system load.
- Trace the high-level CDN and DNS caching flows.
- Distinguish CPU caches, RAM-based caches and durable databases.
- Compare cache-aside and write-through strategies.
- Explain LRU, LFU, TTL and no-eviction behavior.
- Apply caching to queries, sessions, external APIs and rate limiting.

1. What is caching?

Simple definition

Caching is a mechanism that reduces the time and effort required to perform work or retrieve data.

Technical definition

Caching keeps a subset of primary data in a location that is faster or easier to access. The subset is chosen based on factors such as access frequency, likely future use, latency and cost.

Why only a subset?

Fast storage is limited and often more expensive than durable secondary storage. Caching everything would waste capacity and defeat the purpose of keeping only high-value, frequently needed data close to the application or user.

Core objective

Reduce latency, repeated computation, network transfer and load on primary systems.

2. Why large systems use caching

Google Search example

Search processing can involve expensive crawling/indexing/ranking-related systems. Popular or repeated queries can reuse previously computed results. On a cache hit, the result is returned quickly; on a miss, the normal processing path runs and the result may be cached for later requests.

Netflix and content delivery

A streaming service prepares multiple encoded resolutions and serves content through geographically distributed edge locations. A nearby edge server can reduce network latency and buffering compared with always fetching from a distant origin.

Social-media trends

Trend detection can require processing huge volumes of posts and expensive analytics. Rather than recomputing trends for every page view, a system can periodically compute and cache regional results, refreshing them as needed.

General pattern

Caching is especially useful when work is expensive and many users request the same or similar result, or when large content must be delivered repeatedly.

3. Three broad caching levels

Network-level caching

Caches content or lookup results across networked systems. The lecture focuses on Content Delivery Networks (CDNs) and DNS caching.

Hardware-level caching

CPU cache hierarchies (L1, L2 and L3) keep frequently or predictively used data close to processing units, reducing the need to access slower memory.

Software-level caching

Application/backend code uses a cache library or service such as Redis or Memcached. These commonly store key-value data in RAM. The label is about the software interface; the implementation still relies on hardware.

4. CDN: Content Delivery Network

Purpose

A CDN caches content on servers geographically closer to end users. These servers are often called edge servers or edge nodes.

Point of Presence (PoP)

A PoP is a regional location or site containing CDN infrastructure, often multiple edge servers.

High-level request flow

1) A user requests a resource using a URL. 2) DNS resolves the domain and CDN routing directs the request toward an appropriate nearby PoP. 3) The edge server checks its cache. 4) On a hit, it serves the cached resource. 5) On a miss, it fetches the resource from the origin server, returns it and may cache it for later requests.

Origin server

The origin holds the authoritative/original content. The CDN edge stores copies; it is not a replacement for the origin.

Routing factors

The lecture mentions geographic proximity and network conditions as possible factors in choosing an edge location. Actual CDN routing policies vary by provider.

TTL

Time to Live controls how long a cached object may remain fresh/usable before it expires or needs revalidation, depending on the cache policy. After expiry, the edge may fetch or revalidate against the origin.

Typical content

Images, video, static HTML, JavaScript, CSS and other web assets are common CDN candidates.

5. DNS caching

Purpose of DNS

DNS maps human-readable domain names to IP addresses so clients can connect to servers.

Recursive resolver

A resolver, commonly operated by an ISP or a public DNS provider, performs the lookup process on behalf of the client and can cache answers.

Lookup flow on a cache miss

1) The device/browser checks its local DNS cache. 2) If needed, the query goes to a recursive resolver. 3) If the resolver has no cached answer, it queries a root server for a referral to the relevant top-level domain (TLD). 4) The TLD server refers it to the domain's authoritative name server. 5) The authoritative server provides the DNS answer. 6) The resolver returns the answer and caches it according to its TTL.

Multiple cache layers

The lecture describes caches at the operating-system/device level, browser level, recursive-resolver level and, in some cases, authoritative infrastructure. Exact behavior depends on the operating system, browser and DNS implementation.

Why caching matters

Without cached DNS answers, clients and resolvers would repeat more network lookups, increasing latency and load.

6. Hardware caching: CPU cache hierarchy

Hierarchy

The lecture presents CPU registers/processing units, L1, L2 and L3 caches, RAM (main memory) and secondary storage as a hierarchy with different speed and capacity characteristics.

L1 / L2 / L3

L1 is typically the smallest and fastest CPU cache, L2 is generally larger and slower than L1, and L3 is often larger and may be shared among cores. Exact organization differs by processor.

Locality

Caches work well because programs often reuse recently accessed data (temporal locality) or access nearby memory locations (spatial locality).

Array example

Sequentially traversing an array often benefits from spatial locality: nearby elements are likely to be fetched together into cache lines, making sequential access efficient.

Why RAM is not the whole solution

RAM is fast but limited and volatile. Secondary storage is slower but retains data when power is off. The two serve different purposes.

7. In-memory databases and key-value stores

What they are

Technologies such as Redis and Memcached are commonly used as in-memory caching systems. The lecture describes them as in-memory key-value stores: a key identifies a value, which may be a string, number, list, JSON-like structure or another supported type.

Why fast

Frequently accessed values are served from RAM rather than requiring a round trip to a disk-based relational database for every read.

RAM trade-offs

RAM offers low-latency access but has limited capacity and is volatile. It is therefore not automatically a durable replacement for PostgreSQL or another persistent database.

Persistence nuance

Some in-memory systems offer persistence or recovery mechanisms, but these are product/configuration-specific. Do not assume every cache is durable or that a cache's persistence behavior is identical to a relational database.

Managed service example

The lecture mentions AWS ElastiCache as a managed caching service.

8. Cache hit and cache miss

Cache hit

The requested key or object is present and usable in the cache. The system can return it without repeating the expensive origin/database/computation work.

Cache miss

The requested data is absent or unusable/expired. The system obtains it from the source of truth or recomputes it, then may populate the cache.

Why measure both

A high hit rate often indicates that the cache is avoiding work, but hit rate alone does not prove the system is healthy. Latency, memory use, stale data, miss cost and eviction behavior also matter.

9. Two caching strategies

Cache-aside (lazy caching)

The application checks the cache first. On a hit, it returns the cached value. On a miss, it reads from the primary database/source, places the result in the cache and returns it. Data is loaded only when requested.

Advantages of cache-aside

Simple, loads only requested data and allows the application to decide which data is worth caching.

Cache-aside risks

The first request after expiry or eviction pays the miss cost. Updates can leave stale cache entries unless the application invalidates or refreshes them. Concurrent misses can cause a cache stampede.

Write-through

When the application changes data, it updates the primary database and the cache in the same application flow. Later reads can use the refreshed cache value.

Write-through trade-off

The write path does more work and can have higher latency because both systems must be updated. It can reduce stale reads when implemented correctly, but failure ordering and partial failures still need explicit handling.

Important clarification

The lecture describes write-through as keeping cache and database fresh together. In real systems, consistency depends on write ordering, error handling, retries and whether both writes succeed; it is not automatically guaranteed.

10. Eviction policies

Why eviction is needed

Cache memory is finite. When it fills, the system needs a rule for removing entries so new values can be admitted.

No eviction

Do not automatically remove entries to make space; a write that needs additional memory may fail or be rejected, depending on configuration.

LRU — Least Recently Used

Evict the entry that has gone the longest without being accessed. It assumes recently used items are more likely to be used again.

LFU — Least Frequently Used

Evict the entry with the lowest access frequency. It favors items that have historically been requested more often.

TTL-based expiration

Give keys a time-to-live. After the TTL expires, the entry is considered expired and can be removed or treated as a miss.

Choosing a policy

Use the access pattern and freshness requirements to guide policy choice. LRU and LFU are different heuristics; neither is universally best. TTL controls freshness/lifetime and may coexist with a memory-eviction policy.

11. Backend use case: database query caching

Scenario

A frequently requested API runs an expensive SQL query involving many joins or aggregations over a large dataset.

Approach

Cache the computed result under a key representing the query and its relevant parameters. On subsequent requests, return the cached result until its TTL expires or the result is invalidated.

Benefits

Reduces repeated database work, database load and API latency.

Invalidation

When underlying records change, the application may delete or refresh affected cache keys. Cache keys must include relevant filters, tenant/user scope and other parameters so one request does not receive another request's result.

Good candidates

Read-heavy, expensive, frequently repeated queries whose results do not need to reflect every write instantly.

12. Backend use case: session storage

Scenario

After authentication, the application issues a session identifier/token and stores session state server-side.

Why a cache

A fast in-memory store can make session lookups inexpensive on each authenticated request and avoid repeatedly querying the primary relational database.

Security note

Session identifiers are sensitive credentials. Use secure generation, transport and cookie settings; apply expiration and revocation; and never expose session secrets in logs.

13. Backend use case: external API caching

Scenario

A backend calls a third-party API, such as a weather API, and uses the response to serve its own frontend.

Approach

Cache the external response for a suitable TTL. Requests during that interval reuse the cached value rather than making another external call.

Benefits

Reduces third-party API billing, avoids rate limits and lowers response latency.

Freshness decision

Choose TTL based on how quickly the external data changes and how stale the product can tolerate it. Real-time or safety-critical information may not be suitable for a long TTL.

14. Backend use case: rate limiting

Purpose

Rate limiting restricts how many requests a client can make during a time window, protecting services from abuse and excessive load.

Redis counter pattern

A middleware identifies a client key (often derived from a trusted client IP or authenticated identity), increments a counter in a fast shared store and checks whether the configured limit has been exceeded.

Example

If the policy allows 50 requests per minute, the middleware counts requests for the client during the current window. Requests beyond the limit are rejected.

HTTP response

The lecture identifies HTTP 429 Too Many Requests as the response when a client exceeds the limit.

Why use Redis

A shared in-memory store can support fast counter operations across multiple backend instances. Storing every request counter in a relational database can add latency and unnecessary load.

Proxy/IP caution

Headers such as X-Forwarded-For are only trustworthy when set/normalized by a trusted reverse proxy and the server is configured with the correct proxy trust boundary. Do not blindly trust a client-supplied header, or clients may spoof their identity.

Atomicity

Increment-and-check operations should be atomic or use an appropriate Redis script/command pattern so concurrent requests cannot bypass the limit.

15. Practical design checklist

Before caching

Identify the expensive operation, request frequency, acceptable staleness, data ownership and source of truth.

Choose the key

Make cache keys deterministic and include every input that changes the result, such as tenant, user scope, filters, locale or query parameters.

Choose TTL

Set TTL according to freshness requirements, not merely convenience. Short TTLs improve freshness but may reduce hit rate; long TTLs improve reuse but increase staleness risk.

Plan invalidation

Decide what happens after create/update/delete operations. Use explicit invalidation, refresh, versioned keys or another deliberate strategy.

Handle failures

Decide what happens if the cache is unavailable: fall back to the database where safe, apply timeouts, and avoid overwhelming the source during a cache outage.

Protect the source

Consider stampede protection, request coalescing, rate limits and bounded concurrency when many requests may miss simultaneously.

Observe

Monitor hit/miss ratio, cache latency, origin latency, evictions, memory use, errors and stale-data incidents.

Security

Do not cache sensitive data under shared keys. Enforce user/tenant boundaries and consider whether data should be cached at all.

16. Interview questions and concise answers

What is caching?

Keeping a useful subset of data in a faster-access location to reduce repeated work and latency.

What is a cache hit?

The requested usable item is found in the cache.

What is a cache miss?

The item is absent or expired, so the system fetches or recomputes it from the source.

Why use a CDN?

To serve cached content from edge locations closer to users, reducing latency and origin load.

What is a PoP?

A Point of Presence: a regional CDN location containing edge infrastructure.

What is TTL?

Time to Live: the duration an entry can remain valid before expiration or revalidation.

How does DNS caching help?

It reuses prior name-resolution results and avoids repeating recursive DNS lookups.

Why not store everything in Redis?

RAM is limited and more expensive than disk storage; caches are usually subsets and are not always durable.

Cache-aside vs write-through?

Cache-aside loads on read misses; write-through updates the cache as part of the write flow along with the primary store.

LRU vs LFU?

LRU evicts least recently accessed; LFU evicts least frequently accessed.

Give four Redis use cases.

Database query/result caching, server-side sessions, external API response caching and rate-limit counters.

Why is Redis useful for rate limiting?

It supports fast shared counters and helps avoid a database round trip on every request.

What can go wrong with caching?

Stale data, incorrect cache keys, cache stampedes, memory pressure, eviction surprises, cache outages and cross-user data leaks.

17. Quick revision sheet

Caching

A useful subset of data kept closer/faster to reduce repeated work.

Hit

Found in cache; return cached result.

Miss

Not found/usable; fetch from source, optionally populate cache.

CDN

Edge caching near users; PoP = regional edge site; origin = original source.

DNS cache

Reuses domain-to-IP answers, subject to TTL.

CPU cache

L1/L2/L3 exploit temporal and spatial locality.

Redis/Memcached

Common in-memory key-value caching technologies.

Cache-aside

Read cache → miss → read DB → populate cache → return.

Write-through

Write primary store and update cache in the write flow.

LRU

Least recently used entry is evicted.

LFU

Least frequently used entry is evicted.

TTL

Entry lifetime / expiration setting.

Use cases

Query results, sessions, external APIs, rate limiting.

Rate-limit response

HTTP 429 Too Many Requests.

Golden rule

Define freshness, key design, invalidation and failure behavior before adding a cache.

Pseudocode: cache-aside read

```
function getResource(key):
cached = cache.get(key)
if cached exists:
return cached // cache hit
value = database.fetch(key) // cache miss
if value exists:
cache.set(key, value, TTL)
return value
```

Pseudocode: rate-limit counter

```
function handleRequest(clientKey):
count = atomicIncrement(counterKey(clientKey, currentWindow))
setExpiryIfNew(counterKey, windowDuration)
if count > allowedRequests:
return HTTP 429
continueToHandler()
```

Implementation note

The increment and expiry setup should be atomic or safely coordinated. The client key must come from a trusted identity source; do not blindly trust arbitrary X-Forwarded-For input.

Suggested hands-on practice

- Start Redis locally and connect using the client library for your backend language.

- Implement cache-aside for a product/detail endpoint with a short TTL.
- Update a product and invalidate or refresh its cache key.
- Add a cached third-party API response and observe how TTL affects request count.
- Store a server-side session with an expiration time.
- Implement a per-client rate-limit counter and return HTTP 429 after the limit.
- Test cache outage, expired entries, concurrent misses and incorrect key isolation.

Important distinctions

- A cache is not automatically the source of truth; the primary database usually remains authoritative.
- RAM is fast but limited and volatile. Some products offer persistence features, but behavior depends on configuration.
- TTL is a freshness/lifetime rule; LRU/LFU are memory-pressure replacement policies.
- Write-through does not guarantee consistency if one of the two writes fails.
- CDN, DNS and CPU caching are different layers that share the idea of reusing data close to where it is needed.

Source boundary

- The transcript gives high-level examples and does not specify the exact internal cache architectures or algorithms used by Google, Netflix or X.
- It mentions Redis, Memcached and AWS ElastiCache but does not provide a complete Redis command reference.
- CDN routing, DNS cache hierarchy and persistence behavior vary by provider and implementation.